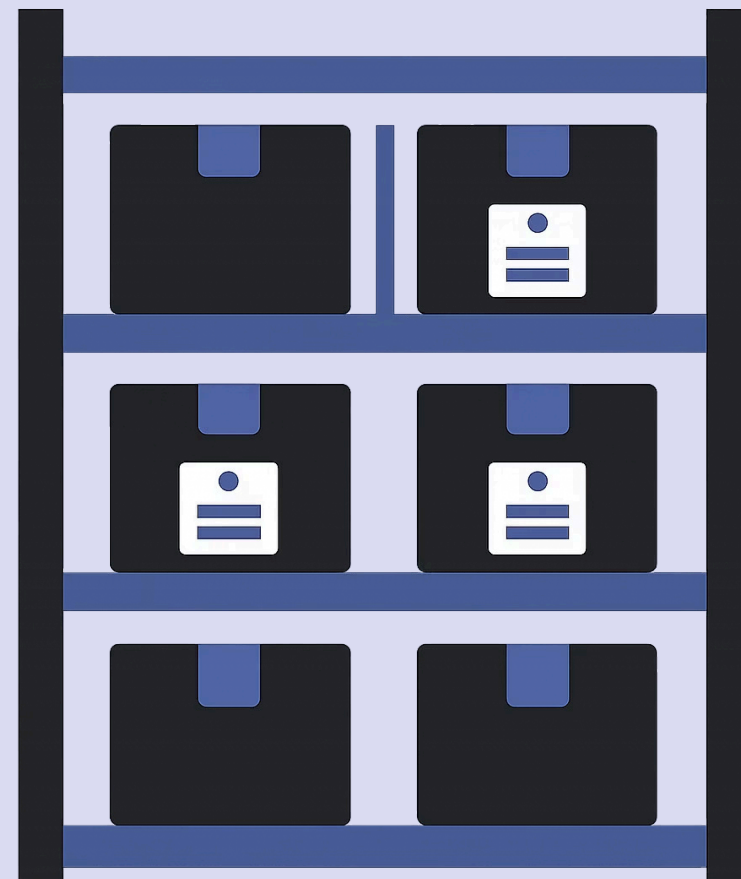


Sistema de Gerenciamento de Estoque para padaria em Python

Um projeto prático que demonstra conceitos fundamentais de Python: Este projeto é um sistema simples de controle de estoque desenvolvido em Python, voltado para o gerenciamento de produtos de uma padaria.

PYTHON INICIANTE

PROJETO PRÁTICO



Estrutura de Dados: Lista de Dicionários

```
produtos = []
```

```
# Cada produto é um dicionário:
```

```
produto = {  
    "nome": "bolo",  
    "preco": 49.99,  
    "estoque": 10  
}
```

Por que lista de dicionários?

foi usado uma lista de dicionários, pois é mais adequada quando a ordem dos elementos importa.

Função de Busca:

Comparação sem Erros

```
def buscar_produto():  
    nome = input("Nome do produto: ").strip()  
    for p in produtos:  
        if p["nome"].lower() == nome.lower():  
            return p  
    return None
```

.strip()

Remove espaços em branco extras digitados pelo usuário antes ou depois do texto.

.lower()

Converte para minúsculas antes de comparar, evitando conflitos entre "Camiseta" e "camiseta".

return None

Retorna `None` se não encontrar o produto, permitindo que outras funções tratem o resultado.



Cadastro e Tratamento de Erros com try/except

```
def cadastrar_produto():  
    nome = input("Nome: ").strip()  
    try:  
        preco = float(input("Preço: "))  
        estoque = int(input("Estoque: "))  
        produto = {"nome": nome,  
                    "preco": preco,  
                    "estoque": estoque}  
        produtos.append(produto)  
        print(f'{nome}' cadastrado!)  
    except ValueError:  
        print("Erro: valores inválidos.")
```

Por que usar try/except?

Sem o bloco try/except, digitar letras em campos numéricos quebraria o programa com um erro fatal. Com ele, o erro é capturado e tratado de forma elegante.



ValueError ocorre quando o usuário digita "abc" onde se espera um número. Sempre proteja entradas numéricas!

Funções de Estoque: Adicionar e Remover

Adicionar Estoque

Usa `buscar_produto()` para localizar o item e soma a quantidade com `+=`. O `try/except` garante que apenas inteiros sejam aceitos.

Remover Estoque

Verifica se o estoque é maior que zero e se a quantidade a retirar não ultrapassa o disponível. Evita valores negativos em estoque.

Editar Produto

Usa o operador `or` para manter o valor antigo caso o usuário pressione Enter sem digitar nada — edição opcional e intuitiva.

```
p["estoque"] += qtd # Adicionar  
p["nome"] = input(...) or p["nome"] # Editar com valor padrão
```

Relatórios: Valor Total e Alertas

```
def valor_total_estoque():  
    total = sum(  
        p["preco"] * p["estoque"]  
        for p in produtos  
    )  
    print(f"Total: R${total:.2f}")  
  
def estoque_baixo():  
    limite = 5  
    baixos = [p for p in produtos  
              if p["estoque"] < limite]
```

Conceitos avançados aplicados



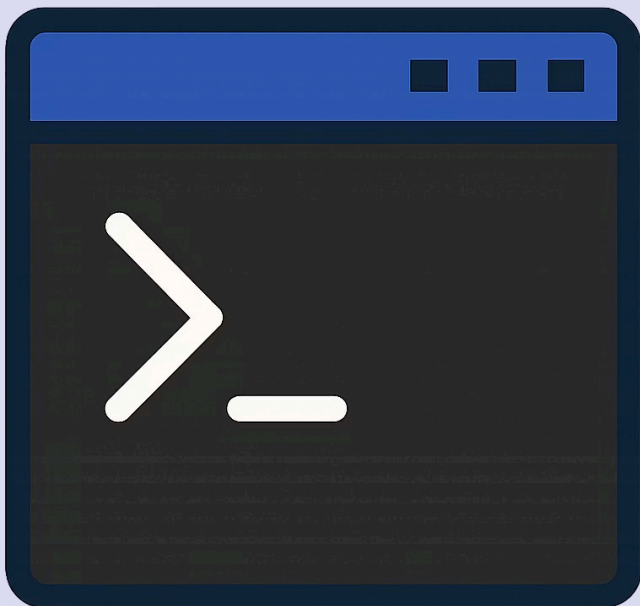
sum() com generator

Calcula o patrimônio total multiplicando preço × estoque de cada produto.



List comprehension

Filtra produtos com estoque abaixo de 5 unidades de forma concisa e eficiente.



Menu Principal: Loop Interativo

```
while True:
    print("1-Cadastrar | 2-Listar | 3-Remover | 4-Editar")
    print("5-Buscar | 6-Add Estoque | 7-Rem Estoque | 0-Sair")
    opcao = input("Escolha: ")
    if opcao == "1": cadastrar_produto()
    elif opcao == "0": break
```

1

while True

Loop infinito que só termina com `break` na opção 0.

2

Estrutura if/elif

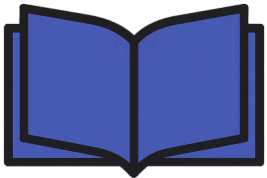
Cada opção chama sua respectiva função, mantendo o código organizado.

3

Opção inválida

O bloco `else` final trata entradas inesperadas sem travar o programa.

Resumo dos Conceitos Aplicados



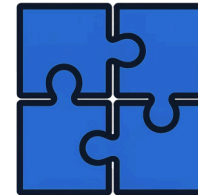
Listas de Dicionários

Estrutura ideal para coleções de objetos com atributos nomeados e ordem preservada.



try/except

Proteção contra entradas inválidas, tornando o programa robusto e confiável.



Funções Reutilizáveis

`buscar_produto()` é chamada por múltiplas funções, evitando repetição de código.